WMTM overview: the design as built, how it was judged, what is done and what is left

For Ben. Written by ProtomegaTron for hive task t-2026-11-13-wmtm-overview-doc, 2026-11-13. Base: wmtm `main` at e59ab57 (GitHub bgoertzel-sing/wmtm).

Summary

WMTM was built as the bounded, mutable working layer described in the design doc of 5 September 2026 (`docs/wmtm_design_doc.txt`). Recalled notes and derived conclusions sit in it together, with attention values, and are forgotten again. The bookkeeping part of the design is in place and passes tests taken from the design's own §12: the bounded store, decay, a forgetting log keyed on the derivation, one eviction rule on every path, and derived items that carry their sources. The reasoning part differs from the design in one major way. The Python PLN engine of §3.4 was removed in October (Track L). The only reasoner now is **PeTTaChainer**, which runs in a background "shadow" worker over atoms produced by Man Hin's semantic parser. Its conclusions are admitted under strict gates. Only deduction-style chaining (Inheritance / Implication transitivity and Member x Inheritance) plus an induction fold is exercised. Abduction, analogy, contradiction handling and evidence aggregation in the §9.1 sense are not present. **Omega-PLN is not used anywhere in wmtm.** Truth values are PeTTaChainer's (strength, confidence) pairs.

The goal chainer runs as a log-only goal engine. Its core is MesTTo's GoalChainer v2, run in a separate process. It is off by default and never acts: it writes proposals to a log, and with an extra flag can show at most three "suggestions (not instructions)" lines. "Goal-driven guidance" in the reasoning sense is narrower. The shadow ranks the chaining goals it asks PeTTaChainer by request overlap, depth, informativeness and source STI (Track N, which is Track M's Design A).

Evaluation is evidence-first. Unit tests come with negative controls that must fail for the stated reason. A/B replays run on frozen parses and are judged by andy's count rule (no more wrong, no fewer correct). Further checks are Mesto's §11 per-conclusion diff, the W1 cross-note rule, a trivial-conclusion filter and a shared parse budget, and every result is re-run by an independent reviewer. No note text leaves pop-os. The latest parser measurement (t-2026-11-12-wmtm-parser-master-adopt-v2) proved 105/105 goals, 97 correct / 4 wrong / 4 unclear. It still misses the deploy bar against live (2C/2W), because 82 of the 105 are restatements and the 4 wrongs exceed live's 2.

Still open, each tied to a task in §3:

- promotion of derived beliefs to LTM (off by default)
- a useful-vs-trivial signal
- inference breadth (abduction, analogy, aggregation)
- graph-only recall as a default
- extending the trivial filter to restatements
- the rules the parser and reasoner are missing (for Man Hin and Roman)

0. Sources, pins and conventions

- **Reference design:** `docs/wmtm_design_doc.txt` (Ben / ProtoCosmo2, 5 Sep 2026), 15 sections. A note added on 16 Oct (Track L) at the top says the Python PLN of §3.4 and §5.3 "was removed" and those sections are historical.
- **As-built docs:** `docs/RUNTIME.md` (v0.7.0-rc2), `docs/GOALS_V2.md`, `docs/GOAL_LIFECYCLE.md`, `docs/GOALCHAINER_DESIGN.md`, `docs/TRACK_M_GOALS.md`, `docs/STOPGAPS.md`, `docs/RELEASES.md`.
- **Code:** package `wmtm/` (about 13,000 lines of Python across `wmtm/*.py` and `wmtm/goals/*.py`). Tests: 66 `test_*` files in `tests/`.
- **Pins:** PeTTaChainer b6aacc68, PeTTa 3681b9b. Man Hin's semantic parser (semantic-parsing-hitl, "sph") is at 24e37182 in the latest task, and 533c2274 before that.

- **Task evidence:** `reports/<task-id>/` in the repo. The one-line outcomes are from the hive record (the order's attachment `wmtm-task-outcomes.md`, cos, 2026-11-13). The parser-master-adopt-v2 report is on branch `hive/t-2026-11-12-wmtm-parser-master-adopt-v2` (785baf3) and not yet on main.
- **Conventions:**
  - `file:line` refers to main e59ab57, unless a report's own commit is quoted.
  - **[unverified]** marks a statement I took from a report or the record and could not re-check myself. For example, the live bot's flags file is not reachable from my sandbox.
  - This document contains no note text: only ids, counts and rule shapes.

1. Design as built

Each part below names the section of the design doc it implements, what the code does, and how it differs from the design. Line numbers in this section come from the design-gap-check report (`reports/t-2026-11-10-wmtm-design-gap-check/REPORT.md`, §1). That report measured main at 27c1a41. Where later tasks changed a part, the task is named.

1.1 Memory model (design §3.1, §3.2, §6, §13.1)

Design	As built	Deviation
§3.1 bounded WMTMStore, capacity 200	`store.py:17-39` validates capacity. `store.py:43-87` admit: on overflow the lowest-STI item is evicted (`:132-144`), and the current request item goes last	Same shape. Capacity is a config value
§3.2 WMTMItem with source type, derived_from, utility	`item.py:16-46`: source_type is recalled / derived / asserted / user, plus `derived_from`, `utility` and `provenance`	Adds `asserted` and `user`, and adds `provenance` (reasoner, goal, stv, proof, run_ts)
§13.1 ephemeral, rebuilt per session	State is a JSON snapshot in the bot's state dir. The forgetting log persists only inside that snapshot (`orchestrator.py:96-121`)	Kept across turns of one bot rather than rebuilt per session. The design's `WMTMDerivedForgotten` LTM note does not exist (GAP-8)
§3.3 / §5 LTM = petta-memory archive	`recall_bridge.py` parses the journal's MemoryClusters, EvidenceFor, PromotesFrom and Supersedes	As designed

1.2 Attention, admission and eviction (design §3.6, §7, §10)

- **STI decay.** `attention.py:35-39` multiplies STI by 0.90 per tick. The design has 2 % per tick (§10 `sti_decay_rate` 0.02), so decay is about five times faster. The floor is 0.01 (`store.py:115-128`). Eviction thresholds are 0.05, and 0.10 for derived items (`forgetting.py:42-49`).
- **Age rule.** `forgetting.py:51-58`: max_age 50, derived 20, min_utility 0.5. The design has 50 / 30 / 2.0. A wall-clock staleness rule was added (`forgetting.py:70-89`, `context.cleanup_stale_items`).
- **Capacity pressure (§7.1c). Not as designed (GAP-3).** Store overflow is STI-only, and policy overflow is utility-only (`forgetting.py:61-67`). The composite STI·0.4 + utility·0.4 + LTI·0.2 and the 1.5× derived penalty are absent. Derived items are instead held to stricter thresholds (`forgetting.py:91-101`).
- **One rule on every path.** Since t-2026-11-06-wmtm-f1-memory (Mesto §9), admit, evict and forget on the user path, the automated path and replay all go through one eviction rule (`test_f1_s9_one_rule.py`, `test_f1_s9_replay.py`).
- **Forgetting log (§13.4).** `forgetting_log.py:42-131`. The key is the derivation (the conclusion plus its sorted sources), as Mesto asked in F1 §4, not the conclusion text alone. There is a FIFO cap of 500. Mesto §4 holds: forgetting frees space but never forbids a conclusion that has a new derivation.

- **Contradiction (§7.1d, §9 contradiction_penalty). Not implemented (GAP-6).** No code handles contradictions.
- **LTI (§3.3 step 2, §8.1c). Not implemented (GAP-10).** LTM clusters carry no LTI, and there is no LTI-driven recall sampling. `AttentionValue.lti` exists, but only WMTM consults it (`attention.py:71`).
- **Utility (§3.5).** `utility.py:27-33` computes uses × 2 + inferred_from × 1.5 − misses × 0.3. A "use" is credited when the item is shown in the context block (`api.py:337-339`) or when its words overlap the response (`context.py:316-331`). A displayed trivial restatement therefore earns the same credit as a useful conclusion (GAP-5).

§12.3 simulation result (design-gap-check, 100 cycles, capacity 20): occupancy stays at the cap (mean 19.8), and all forgetting comes from capacity overflow. The decay, age and utility triggers evicted 0 items. All 8 useful notes survived, while background notes lived 3.3 cycles on average. The memory is stable and selective, but the selection comes from STI reinforcement on use, not from the design's composite.

1.3 Recall (design §3.3, §8, §13.5)

- **Keyword relevance.** `recall_bridge.py:229-302` scores IDF overlap, substrings, bigrams, evidence count and recency. The per-turn budget was top_k 15 (`api.py:253`) until t-2026-11-11-wmtm-recall-budget-10 set `recall_budget=10` on every recall path, as the design's §8.3 asks.
- **Spreading activation.** `recall_bridge.py:386-473` spreads over EvidenceFor in both directions, at depth 2, with decay 0.3 (the design has 0.7). At 27c1a41 the spread only reordered clusters that already had a keyword hit (`:262-263`, GAP-1), and PromotesFrom was parsed but not traversed. t-2026-11-10-wmtm-graph-recall made recall spread over EvidenceFor without keyword overlap and over PromotesFrom at depth 1. The floor 0.1 and the PromotesFrom weight 0.5 came in with t-2026-11-10-wmtm-graph-recall-floor-replay.
- **Graph-only recall is not a default.** t-2026-11-11-wmtm-graph-only-probe found that it hurts admission precision (B 14 % useful vs A 56 %; 471 vs 32 admits) and churns memory. t-2026-11-11-wmtm-spread-seeds-probe found that seeding only from this turn's matches (arm E) removes the churn (0 evictions vs 243, precision 0.52 vs 0.15), but then nothing graph-only is admitted on keyword-free turns. The choice of default is open (§3).
- **Superseded clusters are never recalled.** They are dropped at parse time (`recall_bridge.py:77-86`, `:127-160`). This is a passing §12.4 property.

1.4 The parser path: from notes to atoms (design §9.3; replaces the design's "lightweight triple parser")

The design's §9.3 asked for an "intentionally simple" subject-predicate-object extractor, with full NLP out of scope. The build went further. Notes are parsed into PLN-style atoms by **Man Hin's semantic parser** (semantic-parsing-hitl, "sph"), called through the bot's gateway. This is the largest deviation from the design. It brings better structure at the price of LLM cost, run-to-run variance, and dependence on what the parser emits.

- **Feed (Track H, `semparse_feed`, `docs/RUNTIME.md`).** Each cycle the top-K active items are queued by content hash. A detached worker parses them with the pinned sph prompt, load-checks every atom in PeTTaChainer and caches the accepted atoms (`semparse_cache/cache.jsonl`). The same content is never parsed twice. A preamble (`wmtm/semparse_preamble.py`, Track M) is prepended to the pinned prompt, and each parse records its `prompt_sha`.
- **Modes.** The live bot parses in legacy mode (t-2026-11-10-wmtm-flag-semparse-legacy; t-2026-11-10-wmtm-gateway-callers: the vendored rc2 4d2ab315 predates `semparse_mode`) **[unverified: live flags]**. t-2026-11-10-wmtm-split-parser-modes-v2 found that in the split setup both modes give a byte-identical live context. Legacy keeps more shadow evidence (478 vs 184 facts; 41 vs 0 goals, all proved) at about 1.8× lower token cost.
- **Parse budget.** There are at most 20 parse attempts per rolling hour (`parse_cap_per_hour`). On top of that, `wmtm/parse_budget.py` gives fixed shares (andy, 2026-11-10): live 10/h and all

hive tasks together 10/h, counted across a host-wide ledger. Each call writes a ledger row before it is sent (t-2026-11-10-wmtm-shared-parse-count-v2). Prompt caching was broken by the OpenClaw gateway, not by wmtm (t-2026-11-10-wmtm-parse-caching). That was patched on pop-os, and the patch is guarded by a daily check (t-2026-11-10-wmtm-gateway-patch-pack, t-2026-11-11-wmtm-gateway-patch-guard).

- **Parse-quality gate** (`wmtm/parse_gate.py`). Before facts enter the reasoner's KB, the gate rejects facts below confidence 0.3, placeholder (pronoun-like) subjects, negative quantifiers, broad and hedged terms (reject by default), and identifiers (hash, uuid, path) typed as categories. It caps conclusions at 3 per fact (`gate_max_per_fact`).
- **Lowering to atoms and its stopgaps.**
  - Narrow pre-load (`semparse_adapter.py:508-515`, STOPGAP, t-2026-11-10-wmtm-narrow-preload). PeTTaChainer refuses a narrowed class inside a statement. wmtm rewrites it to one head-noun link per symbol before loading, at 0 extra parse calls. On the three frozen runs this recovered 76/88/75 of 99/109/101 refused statements and took B from 0 to 12/14/10 goals, all proved.
  - Normaliser (`wmtm/parse_normalise.py`, t-2026-11-10-wmtm-normaliser-v2). This is in, but it changed nothing on the parsers of that day: 533c2274 and f44d7b74 emit only Member facts, with no flat rules and no Inheritance.
  - Class-level statements. t-2026-11-09-wmtm-flat-inheritance-v2 found them absent from parser output (0 Inheritance atoms). The fix belongs to the parser (Man Hin) or to a wmtm-side fallback. Undecided.
  - Master loader (t-2026-11-12-wmtm-parser-master-adopt-v2, sph 24e37182). This loads the parser's output together with a seeded rule file of 280 rules. It is on the task branch only. The live `build_snapshot` does not use it.
  - Names and skolems. `name_resolve.py` makes a named individual one entity across notes (Mesto §1). `sk_registry.py` tracks `sk_*` placeholder constants, which are never goals unless `shadow_skolem_goals` is set.
- **Variance.** At a fixed parser version and prompt, all 20 evaluation notes parse differently across 3 fresh runs (atom totals 810/896/842, about 10 % of the mean; t-2026-11-10-wmtm-parse-variance). This is why evaluation uses **frozen parses** (`wmtm/frozen_parse.py`, keyed on note hash, parser pin and prompt sha). Sentence-by-sentence parsing was tried (t-2026-11-11-wmtm-sentence-parse-v3). It showed no clear coverage gain, was more stable, and was less reliable (provider refusals). Its conclusions were all trivial echoes (t-2026-11-12-wmtm-sentence-reasoning).

1.5 Reasoning through PeTTaChainer (design §3.4, §9; §3.4 and §5.3 historical since Track L)

- **Worker** (`wmtm/pc_shadow.py`, `wmtm/workers/`; Track G, `pc_shadow`). Each cycle, rate-limited to one launch per 300 s, a niced detached worker gets the top-K active items. It builds a small PLN KB **from cached parses only** (since v0.4 there is no regex fallback, and unparsed items add no facts). It applies the parse gate, asks PeTTaChainer a ranked set of chain goals under a hard per-query timeout, and writes `pc_shadow/shadow.jsonl` and `admit.json`. A **canary** (the Grace example, which must be proved) and a **negative control** (which must not be proved) run through the same path every time. If either goes wrong, the run is `canary_fail` and nothing from it is trusted.
- **Admission** (`wmtm/pc_admit.py:163` `load_candidates`, `orchestrator.py`). On each turn WMTM reads `admit.json`. It admits only if the file is fresh (≤ 900 s), the status is ok, the canary passed and the negative control was not proved. Each result must also pass all of these:
  - It is a real derivation. `is_derivation` (`pc_admit.py:34`) rejects proofs that are just an asserted fact or a revision of asserted facts.
  - Its strength is ≥ 0.5. Lower means "probably not": the conclusion is dropped, or admitted as `unlikely: …`.
  - At least one supporting item is still active.

- It is neither already known nor in the forgetting log.
- It fits the caps: 3 per fact and 5 per cycle.

Admitted items get source_type `derived`, `derived_from` = the supporting item ids, initial STI = strength × confidence, and a `provenance` record (reasoner, goal, stv, proof, run time). The cycle never waits for the worker.

- **Trivial filter** (`pc_admit.py:455` `trivial_reason`, t-2026-11-10-wmtm-trivial-filter-followup-v2). This drops conclusions that only echo a compound name of their own source (X_Y → Y) and detects the nwdef shape exactly. It keeps 49/49 non-trivial conclusions on the real-data control. It does **not** yet catch restatements: on the master-adopt-v2 run it caught 0 of 82 (77 symmetry, 5 coreference).
- **Which inference patterns are real (design §9.1).**

§9.1 pattern	Status	Source
a) deduction	**Real.** Inheritance / Implication transitivity and Member × Inheritance chains, up to `shadow_max_hops`	`docs/RUNTIME.md` "Goal ranking"; design-gap-check §1
b) induction	**Real but adds nothing yet.** It folds class links over instances (implication-instance-fold, mass-weighted average as §9.1b says). On frozen parses: 0 new conclusions at min_shared=3, one UNCLEAR rule at 2	t-2026-11-11-wmtm-induction (review 3)
c) abduction	**Not present**	design-gap-check GAP-9
d) analogy	**Not present**	GAP-9
e) evidence aggregation	**Partly.** PeTTaChainer's merge/revision of identical facts exists, but no noisy-OR across conclusions	`pc_admit.is_derivation`; GAP-9
contradiction handling	**Not present**	GAP-6

- **Upstream stopgap.** PC-EQ1 (`pc_shadow.py:182`, `docs/ST0PGAPS.md`): Equivalence is not passed to PeTTaChainer, because its add is rejected upstream.
- **Writeback.** PeTTaChainer-admitted items never reach the journal unless `reasoner_writeback` is set (default false). Since Track L these are the only derived beliefs, so by default WMTM adds nothing to LTM (GAP-4).

1.6 Truth values, PLN and Omega-PLN: what is real

- **Truth values.** Every derived item carries PeTTaChainer's simple truth value (strength, confidence) in `provenance.stv`, together with the proof term. Mesto §3 requires a conclusion to keep its engine truth value and sources and never be re-parsed. That holds, and is tested in `test_pc_shadow.py::test_derived_enters_kb_with_engine_tv_and_sources` and `::test_derived_dropped_when_a_source_is_withdrawn` (design-gap-check §4). Initial STI is strength × confidence. For admission, strength is read as a probability (below 0.5 = negative).
- **PLN.** The PLN that runs is **PeTTaChainer's rule set** (pinned b6aacc68, on PeTTa 3681b9b), in the chain shapes listed in §1.5. The design's §3.4 Python WMTMInferenceEngine was removed in v0.5 (Track L, `docs/RUNTIME.md` "Reasoner switch"). `reasoner=python` now means off, and there is no fallback.
- **Omega-PLN. Not present in wmtm.** A case-insensitive search for `omega-pln`, `omegapln` and `omega pln` over the code, docs and reports at e59ab57 finds no match. As the order notes, MeTTaDinC's OQ 6 found that the OmegaPLN reference interpreter does not exist yet **[unverified by me: taken from the order]**. So neither higher-order truth values nor per-packet provenance in the Omega-PLN sense are implemented. Provenance is the proof term plus `derived_from` ids.
- **Rule coverage is the binding limit.** Of 75 conclusions a reader expected on the master-adopt-v2 notes, B reached 8. 33 had a rule but were not derived (32 because the

premise was not parsed, 1 with the premise loaded but not derived). 34 need rules that don't exist (hour ordering 7, supersession 6, sums 5, contradiction 4) (t-2026-11-12-wmtm-parser-master-adopt-v2, reader-expectation pass).

1.7 The goal chainer and goal-driven guidance of Omega

"Goal-driven" means two different things in wmtm, and they are kept apart here.

(a) Goal ranking inside the reasoner (Track N, rc2; `docs/RUNTIME.md` **"Goal ranking").** PeTTaChainer is backward-chaining, so it needs goals. The shadow builds a pool of goals that are derivable by Inheritance / Implies transitivity and Member + Inheritance chains, up to `shadow_max_hops`. Goals already asserted are left out, and so are `sk_*` placeholders by default. Each goal is scored as `w_req`·(overlap with the current request) + `w_depth`·(hops − 1) + `w_info`·(category informativeness) + `w_sti`·(max source STI), default weights 2, 1, 1, 0.5 (`runtime_config.py:158`), and the top goals are asked. This is the only way the current conversation steers inference. Track M (`docs/TRACK_M_GOALS.md`) has three parts. Its Design A (request anchor + depth + informativeness + STI) is exactly this ranking, so it **is built**. Design B (goals from open questions; the parser's queries are still dropped) and the one-line contradiction check are **not built**.

(b) The goal engine: GoalChainer (v0.6 native, v0.7 = GoalChainer v2). Its design is in `docs/GOALCHAINER_DESIGN.md` (a proposal for Ben, step 4 of the 2026-10-12 plan). The as-built version is in `docs/RUNTIME.md` "Goal engine" and `docs/GOALS_V2.md`.

- **What WMTM does.** WMTM extracts open items from memory: explicit markers, resolved / superseded detection, replies in the request history, addressees from `speaker_map` only, and the bot's own promises as self-tasks (`wmtm/goals/needs.py`, `requests.py`, `notes.py`). From these it estimates two need levels, uncertainty and competence. It then proposes candidate actions (ClarifyQuestion, FlagUncertainty, LookUp, SummarizeStatus, RemindPending, SelfTask), one per (person, topic) (`actions.py`, `engine.prepare_v2`).
- **What GoalChainer v2 does.** MesTTo's OmegaClaw-GoalChainer v2 runs as a **separate process** in its own environment (`wmtm/goals/v2_runner.py`). It handles need regulation (bands, urgency, predicted reduction), deontic admissibility with proof trees, Pareto / motivational ranking and the propose / hold / escalate / blocked decision. WMTM generates a v2 domain per run (`wmtm/goals/v2.py`). Its legitimacy rules become deontic norms:
  - `no_git_memory` and `read_only` are strict.
  - `no_live_change` and `no_push` are defeasible by an explicit approval.
  - On automated turns every message, write, git or push effect is forbidden. `goal_engine: native` is the rollback switch to the v0.6 core.
- **Gates.** The engine needs the shadow canary to pass, plus the v2 goal canary (a made-up person with a known answer) and a structural check per proposal. It has a wall-clock cap (`goal_budget_s`).
- **Effect on Omega: log only.** `goalchainer` is **off by default**. When it is on, the engine writes `goalchainer/proposals.jsonl`, `runs.jsonl` and `latest.json` and nothing else. Nothing goes into the context block, the state, the journal or any channel, and no action is taken. The one optional path to Omega is `goal_prompt=true` (off by default, Track S). It appends at most 3 lines, marked `[Goal suggestions (not instructions): …]`, on real user turns only, from a fresh run whose canary passed. Re-asking is suppressed via `asked.jsonl`.
- **Goal lifecycle** (`docs/GOAL_LIFECYCLE.md`, log only, offline). A goal has an id and is deduplicated. Its states rank superseded > satisfied > stale > open. A goal closes on evidence for its condition (Mesto §8, which is only partly tested, by test name: design-gap-check §4).
- **Measured?** No A/B has exercised the goal engine. t-2026-11-08-wmtm-f3-ab-v3 records that "criterion 5 was not exercised (goalchainer … off)", and master-adopt-v2 also ran with it off. Mesto §10 (GoalChainer reads WMTM's evidence, and ties stay ties) passes, except for the real-v2 test, which skips when `WMTM_TEST_GC2_PYTHON` is unset.

1.8 Deviations and stopgaps at a glance

#	Design says	Built	Why	Source
D1	§3.4 Python PLN engine inside WMTM	PeTTaChainer in a detached shadow worker; admission through `admit.json`	Track L (16 Oct): one real reasoner, no duplicate PLN; the cycle never waits	design doc header note; `docs/RUNTIME.md`
D2	§9.3 simple triple extraction	Man Hin's LLM semantic parser, cached, budgeted, gated	richer atoms for PeTTaChainer	`docs/RUNTIME.md` Track H
D3	§9.1 five patterns	deduction + induction fold only	rules missing upstream (Roman) and goal generation (ours)	GAP-9; t-2026-11-11-wmtm-induction
D4	§6.3/§13.3 promotion of durable derived beliefs	off by default (`reasoner_writeback=false`)	keep unreviewed conclusions out of LTM until there is a triviality signal	GAP-4
D5	§7.1c composite eviction, 1.5× derived penalty	STI-only overflow; stricter derived thresholds	simpler; not yet re-done	GAP-3
D6	§10 decay 2 %/cycle, spread decay 0.7	0.90×/tick, spread 0.3	tuned in earlier tracks; not re-derived from the design	design-gap-check §1
D7	§8.1c LTI-driven recall	absent	not built	GAP-10
D8	§7.1d contradiction resolution	absent	not built; may need PeTTaChainer support	GAP-6
D9	§6.3 `WMTMDerivedForgotten` LTM note	forgetting log only in the WMTM snapshot	not built	GAP-8
D10	§12.2 writeback as new EvidenceFor links	EnrichmentNote after 30 cycles of use	code's form differs	GAP-7
D11	— (not in design)	GoalChainer goal engine, log-only	Ben's 2026-10-12 plan step 4	`docs/GOALCHAINER_DESIGN.md`
S1	—	**STOPGAP PC-EQ1**: Equivalence not passed to PeTTaChainer	upstream rejects the add	`pc_shadow.py:182`; `docs/STOPGAPS.md`
S2	—	**STOPGAP narrow_preload**: narrowed classes rewritten to head-noun links	PeTTaChainer refuses narrowed classes	`semparse_adapter.py:508`; t-2026-11-10-wmtm-narrow-preload
S3	—	gateway prompt-cache patch on pop-os, with a daily guard	OpenClaw gateway put per-request text before the cache boundary	t-2026-11-10-wmtm-parse-caching; t-2026-11-11-wmtm-gateway-patch-guard

2. How wmtm was tested and judged

2.1 What "working" means

andy's decision of 2026-11-10 (order attachment `SCORING-RULES.md`): "working" is judged by **the design doc's §12 tests and Mesto's worked examples**. The A/B count rule is a no-regression guard, not a definition of success. t-2026-11-10-wmtm-design-gap-check turned both into tests (`tests/test_design_s12_gapcheck.py`, `tests/test_design_s12_engine.py`). At 27c1a41 the result was 16 pass + 8 xfail in the gap-check file and 2 pass in the engine file. At e59ab57 the gap-check file gives 19 passed + 5 xfailed (GAP-3, 4, 6, 7, 8): later merged tasks closed GAP-1,

GAP-2 and the S part of GAP-5 (the GAP-5 xfail was removed in 8849379, trivial-filter, PR #21). Each xfail names its GAP id, so closing a gap flips a test.

2.2 Unit tests with negative controls

- `tests/` has 66 `test_*` files at e59ab57. They cover the store, the forgetting log and one-rule eviction (F1), context goals (F2), the shadow and admission gates, the parse gate, the narrow pre-load, the normaliser, frozen parses, the parse budget and the goal engine.
- Every feature task had to show a **negative control**: a case that must fail, and must fail for the stated reason. The same idea runs in production. Each shadow run carries the Grace canary (must be proved) and a negative control (must not be proved). Each goal-engine run carries a known-answer goal canary (`docs/RUNTIME.md`). A failed canary voids the whole run instead of being averaged in.
- Exact truth values are asserted where they matter. For example, the kbA/kbD/kbH tests check exact stv (t-2026-11-08-wmtm-f3-ab-v3, commit 1571d80).

2.3 Frozen parses and A/B replays

- Parser output varies run to run (about 10 % of atoms; §1.4). For that reason the reasoning comparisons replay **frozen parses** (`wmtm/frozen_parse.py`): the same note hashes, parser pin and prompt sha in both arms. A difference between arms is then a reasoning difference and not parser noise.
- **A** is the live configuration, and **B** is the candidate. The F3 series (t-2026-11-07-wmtm-f3-ab → -v2 → -v3) set the baseline A_v3 = **2 correct / 2 wrong**. Later tasks compare against that baseline.
- **Count rule** (pre-registered in t-2026-11-07-wmtm-f3-ab, item 1): B passes only if correct(B) ≥ correct(A) and wrong(B) ≤ wrong(A). Unclear (U) conclusions are reported but do not count either way.

2.4 Scoring rules for individual conclusions

- **Mesto §11 per-conclusion diff.** Each conclusion is listed by id/hash as kept, gained or dropped between A and B, and every drop is classified as either a real loss or a harmless one (e.g. f3-ab-v3 §11 table).
- **W1, cross-note conflation** (`SCORING-RULES.md`, P4): a conclusion is wrong when its premises come from more than one note and include an `sk_` fact. The DEVIATION_1 nwdef join stays W1 ("keep it wrong", Work msg 285). An identical class-level fact found in two notes is **not** W1. That is corroboration (Work msg 355, 2026-11-09).
- **Trivial vs useful.** t-2026-11-10-wmtm-score-49-goals hand-scored 41 correct conclusions and found 39 of them to be one-hop "an X_Y is a Y" restatements. The trivial filter (§1.5) came out of that. Restatements by symmetry or coreference are now counted separately ("non-restatement" columns in master-adopt-v2).
- **Reader-expectation pass** (master-adopt-v2): a reader lists the conclusions a person would draw from the notes, and B is scored against that list. This measures recall of useful inferences, which the count rule cannot.

2.5 Process controls

- **Parse budget** (§1.4): fixed shares of 10/h for live and 10/h for all tasks, through one ledger, so evaluation can never starve the live bot.
- **Pins in every report:** PeTTa, PeTTaChainer, sph commit, seeded rule-file sha256 (e.g. master-adopt-v2 `armB/pins.txt`). Reports ship `SHA256SUMS` and a `tools/recompute.py`, so a reviewer can recompute the tables.
- **Independent review.** Every task result is reviewed before it closes, and the reviewer re-runs the key numbers. For example, reviewer.claude reproduced the deploy-split probe and checked about 60 citations. master-adopt-v2 closed after review 5. The rule applies to this document too.

- **Privacy.** Reports carry ids, hashes, counts and rule shapes, never note text (e.g. f3-ab-v3 "§11 per-conclusion diff, inline (ids/hashes only)"). The history of `main` was rewritten so that it holds no real bot memories (`docs/RELEASES.md`). Raw parses and states stay on the host that ran the task **[unverified: host layout]**.

2.6 Limits of this evaluation

- **What it can show.** That a change does not regress the judged conclusions on a fixed basis, that each §12.4 invariant holds, that admitted conclusions are real derivations carrying the engine's truth value, and exactly which conclusions were gained or lost (by id).
- **Small and not independent.** The A/B sets are about 20 notes from one bot. The A baseline has only 2C/2W, so one extra wrong conclusion fails the rule. The rule protects against regressions but has little power to show gains.
- **Correctness ≠ usefulness.** Most "correct" conclusions are restatements (82/105 in the latest run). A high C count can hide a reasoner that adds almost nothing. The reader pass (8 of 75 expected) is the more honest number.
- **Hand-scored.** C/W/U and W1 labels are human or agent judgements under written rules. They are reviewed but not double-blind.
- **Offline only.** Frozen-parse replays do not measure what the live bot shows a user. No user-facing outcome study exists, and the goal engine has never been in an A/B (§1.7).
- **Pinned upstreams.** Results hold at the pins. A new parser or PeTTaChainer commit means re-running, as the narrow pre-load and normaliser tasks showed.

3. Status: implemented, measured, open

3.1 Implemented and on main (e59ab57)

Merged task branches, from `git log --merges main`: t-2026-11-04-p2-parser, t-2026-11-04-pettachainer-migration-report, t-2026-11-06-p4-proof-makeup, t-2026-11-06-wmtm-f1-memory, t-2026-11-06-wmtm-f2-context, t-2026-11-07-wmtm-f12-merge, t-2026-11-09-wmtm-flat-inheritance-v2, t-2026-11-09-wmtm-live-path-replay-v3, t-2026-11-10-wmtm-graph-recall, t-2026-11-10-wmtm-shared-parse-count-v2, t-2026-11-10-wmtm-trivial-filter, t-2026-11-11-wmtm-induction, t-2026-11-11-wmtm-spread-seeds-probe (plus the earlier readme-flags and goal-lifecycle-rating branches).

Part	Where on main	Task
Bounded store, decay, one eviction rule on every path, forgetting log keyed on derivation, conclusions keep engine TV + sources	`store.py`, `forgetting.py`, `forgetting_log.py`, `orchestrator.py`	t-2026-11-06-wmtm-f1-memory, t-2026-11-07-wmtm-f12-merge
Context block: parse-equal merging only, TV and sources shown, attention-only ordering, ties escalate	`context.py`, `api.py`	t-2026-11-06-wmtm-f2-context
PeTTaChainer shadow + gated admission (`is_derivation`, min strength, caps)	`pc_shadow.py`, `pc_admit.py`	Track G/L; t-2026-11-06-p4-proof-makeup
Induction goals (implication-instance-fold)	`pc_shadow.py:363` `induction_goal_pool`	t-2026-11-11-wmtm-induction
Trivial filter (X_Y → Y, exact nwdef shape)	`pc_admit.py:455` `trivial_reason`	t-2026-11-10-wmtm-trivial-filter(-followup-v2)
Graph recall without keyword overlap; PromotesFrom depth 1	`recall_bridge.py`	t-2026-11-10-wmtm-graph-recall (+ floor-replay)
`recall_spread_seeds` flag (default `all`)	`context.py:732`, `runtime_config.py:174`	t-2026-11-11-wmtm-spread-seeds-probe
`recall_budget` 10 per cycle	`runtime_config.py:170`, `api.py:261-263`	t-2026-11-11-wmtm-recall-budget-10

Part	Where on main	Task
Fixed parse shares (live 10/h, tasks 10/h)	`parse_budget.py`	t-2026-11-10-wmtm-shared-parse-count-v2
Narrow pre-load stopgap; normaliser; frozen parses	`semparse_adapter.py:508`, `parse_normalise.py`, `frozen_parse.py`	t-2026-11-10-wmtm-narrow-preload, -normaliser-v2, -parse-variance
Goal engine (log only) on GoalChainer v2, suggestions block	`wmtm/goals/`, `goal_prompt.py`	v0.6/v0.7 tracks; `docs/GOALS_V2.md`
Design §12 acceptance suite with strict xfails	`tests/test_design_s12_*.py`	t-2026-11-10-wmtm-design-gap-check

Not on main: the master loader (branch `hive/t-2026-11-12-wmtm-parser-master-adopt-v2`, 785baf3) and the sentence-reasoning probe (`hive/t-2026-11-12-wmtm-sentence-reasoning`). **Live** runs the vendored rc2 4d2ab315, not main, in legacy parse mode (t-2026-11-09-wmtm-live-path-replay-v3 PINS, t-2026-11-10-wmtm-gateway-callers) **[unverified: live flags not reachable]**.

3.2 What has been measured (key numbers)

A = live behaviour, 2C/2W on the judged evaluation notes. C/W/U = correct / wrong / unclear.

Task	Basis	Result	Count rule
t-2026-11-08-wmtm-f3-ab-v3	B = no wmtm-side rules (3288bf2)	B 8C/2W/1U vs A 2C/2W; warm-up and budget hold	holds; deploy: not yet
t-2026-11-09-wmtm-live-path-replay-v3	parser 533c2274 / f44d7b74	B builds 0 goals (reader names Narrow only inside Member); Nov 5 parse: 9 goals, 9 proved	— ; deploy: no
t-2026-11-10-wmtm-score-49-goals	legacy parse	B 41C/8W/0U; 39 of 41 C are one-hop X_Y → Y	fails on wrongs
t-2026-11-10-wmtm-narrow-preload	frozen r1/r2/r3	B 0 → 12/14/10 goals, all proved; 6C/0W, 6C/2W, 5C/0W	holds (judged subset)
t-2026-11-10-wmtm-normaliser-v2	533c2274 / f44d7b74; Nov 5	0 goals before and after; Nov 5: 9/9, 6C/0W/1U	—
t-2026-11-10-wmtm-split-parser-modes-v2	legacy vs catalog	identical live context; legacy 478 vs 184 facts, 41 vs 0 goals, ~1.8× cheaper	—
t-2026-11-11-wmtm-graph-only-probe	graph-only recall	precision 14 % vs 56 %; 471 vs 32 admits; 186 useful evicted	—
t-2026-11-11-wmtm-recall-budget-10	budget 10	339 vs 471 admits, 243 vs 383 evictions; precision 0.15 unchanged	—
t-2026-11-11-wmtm-spread-seeds-probe	arm E (this turn's seeds)	0 evictions vs 243; precision 0.52 vs 0.15; nothing graph-only on keyword-free turns	—
t-2026-11-11-wmtm-induction	frozen parses	0 new at min_shared=3; 1 UNCLEAR at 2	still fails
t-2026-11-12-wmtm-sentence-reasoning	sentence stores	14 conclusions, all trivial echoes, 0 wrong	—
t-2026-11-12-wmtm-parser-master-adopt-v2	sph 24e37182 + 280 seeded rules (full_s2)	**105/105 proved, 97C/4W/4U**; 82 of 105 restatements (77 symmetry, 5 coref), trivial filter catches 0; W1 = 0; non-restatement 22C/1W	**fails** as a whole (4W > 2W); **holds** on the 23 non-restatements

Master-adopt-v2 further: sens_s2 119/119 proved, 110C/4W/5U (fails); Mesto §11: 10 dropped, 4 real losses; reader pass: 75 expected, B reached 8 (33 rule present but not derived: 32 premise not parsed, 1 premise loaded but not derived; 34 need a missing rule). Worker recommendation: do not adopt yet.

3.3 Open, with owner and task

Open item	State	Owner / decider	Task reference
Promotion of derived beliefs to LTM (GAP-4)	off by default; XS code change, waits on a triviality signal	Ben / andy decide	t-2026-11-10-wmtm-design-gap-check
Useful-vs-trivial signal (GAP-5)	S part closed (trivial-restatement filter, 8849379 on main; xfail removed); M part open: utility still credits display	ours (M)	t-2026-11-10-wmtm-trivial-filter; design-gap-check; t-2026-11-10-wmtm-score-49-goals
Trivial-filter extension to restatements; adjective seeding	0 of 82 restatements caught	andy decides	t-2026-11-12-wmtm-parser-master-adopt-v2
Adopt the master loader in live `build_snapshot`	branch only; not recommended yet	andy decides	parser-master-adopt-v2
Inference breadth: abduction, analogy, aggregation (GAP-9); still in scope?	not present	Roman (PeTTaChainer rules) + ours; Ben decides scope	design-gap-check
Missing rules (hour ordering 7, supersession 6, sums 5, contradiction 4)	34 expected conclusions blocked	Roman / Man Hin	parser-master-adopt-v2 reader pass
Parser output: flat Inheritance / class-level statements	absent from parser output	Man Hin, or a wmtm fallback	t-2026-11-09-wmtm-flat-inheritance-v2
Graph-recall default (`recall_spread_seeds`)	default `all`; arm E removes churn but adds no graph-only recall	andy decides	t-2026-11-11-wmtm-spread-seeds-probe, -graph-only-probe
Deploy split: live context gets 0 derived lines	split feasible with existing flags	andy	t-2026-11-10-wmtm-deploy-split
Composite eviction (GAP-3), contradiction (GAP-6), EvidenceFor writeback (GAP-7), forgotten-note (GAP-8), LTI (GAP-10)	not built; strict xfails	ours	design-gap-check §5
Mesto §2 (fact in other words via calibrated link)	no test	ours (+ likely Man Hin)	design-gap-check §4
Goal engine evaluation	never exercised in an A/B	ours	f3-ab-v3 criterion 5
Omega-PLN	not used; reference interpreter does not exist yet	upstream	order; MeTTaDinC OQ 6 **[unverified]**

3.4 Stopgaps to remove when upstream catches up

Stopgap	Where	Remove when
Narrow pre-load rewrite	`semparse_adapter.py` (STOPGAP narrow_preload)	PeTTaChainer accepts narrowed classes inside statements
PC-EQ1 Equivalence not passed	`pc_shadow.py:182`	PeTTaChainer accepts the Equivalence add
Gateway cache patch	pop-os OpenClaw gateway (guard timer)	Fixed upstream in OpenClaw
Seeded rule file (280 rules)	master-adopt-v2 branch	The parser emits the class-level rules itself

Only PC-EQ1 is listed in `docs/STOPGAPS.md`. The narrow pre-load rewrite is marked STOPGAP (`narrow_preload`) in code (`semparse_adapter.py:473`, `:508`) and recorded in t-2026-11-10-wmtm-narrow-preload. The gateway cache patch is recorded in t-2026-11-10-wmtm-parse-caching and t-2026-11-11-wmtm-gateway-patch-guard. The seeded rule file is recorded on the master-adopt-v2 branch and in its report (t-2026-11-12-wmtm-parser-master-adopt-v2).